The expect method behaves the same as allow except that at the end of the spec RSpec additionally and automatically validates whether the expected methods have been called. If not, it fails the spec.

A downside of this mechanism is that the expectation happens at the beginning of the spec, but the validation happens at the end, and only implicitly. This can make the spec hard to read. An alternative is to use a slightly different form of expect at the end of the spec:

```
obj = double
allow(obj).to receive(:cost).and_return("cheap")
obj.cost
expect(obj).to have_received(:cost)
```

The allow and expect constructs in RSpec are powerful. You can even use them on objects that are not test doubles to stub a particular method on an existing object:

```
kermit = Muppet.new
allow(kermit).to receive(:greeting).and_return("Hi ho")
```

A potential problem with test doubles is that the API of the underlying object might change, but the test, with its stubbed method blissfully continues to pass. RSpec offers some protection from that with the instance_double variation. An instance_double call takes a class as an argument:

```
fake_product = instance_double(Product)
allow(fake_product).to receive(:name).and_return("pretzel")
```

Now, when you call allow or expect with the instance double as an argument, RSpec checks to see if the class in question actually defines the method you are stubbing. (There's a similar RSpec creator, class_double for class methods rather than instance methods.) If the method doesn't exist, RSpec raises an error at the point of the declaration.

This only scratches the surface of RSpec's mock package.

What's Next

In this chapter, we covered Ruby's two most commonly used test frameworks: minitest and RSpec. Which should you use? Well, if you are working on a project that already uses one of them, we recommend sticking with that one. There's not so much difference between the two that its worth re-writing all your tests.

If you are starting a new project, consider whether you like RSpec's syntax. RSpec is probably more widely used, but there are still some very prominent Ruby projects that use minitest, including Ruby on Rails. RSpec has a higher initial complexity, but is also more flexible and has more available functionality out of the box. Ultimately, though, it's a question of which syntax you like better and will get you to write more tests.

We've finished our tutorial of the Ruby language, and now its time to widen our view and take a look at the larger Ruby tool ecosystem.